Line Following Robot Report

Authors: Fynn Morris, Oliver Hamilton, Ara Jolly, Saskia Nyhoff

Group: LFR LabA/03 F

Student IDs: 15967490, 59808489, 35845194, 23589059

Course Code: ENMT221

Date: 9/10/2026

Table of Contents

Abstract 3
Introduction and Constraints 4
Design Evaluation 4
Initial Design 4
Boards 4
Cad modelling and Motion Transmission 6
Laser Cutting 19
Software and Code 20
Robot Performance 21
Evaluation Against Requirements 23
Unique features 24
Comparison with Other Groups 24
Group Reflection 26
Conclusion 27
References 28
Appendices 29

Abstract

This report describes the design, development and testing of a line following robot. This robot was built to follow a white line on a black surface and was made aiming to complete the course within a personal goal of 15 seconds. The design started by incorporating group members' individual PCB, chassis and motion transmission designs and was adapted and modified as the project progressed.

The robot used an ATmega4808 microcontroller board containing the motor driver connections and five TCRT5000 sensor boards. The motors were initially going to have electrical braking, but this was replaced by rubber band braking which was quicker and easier to implement.

The motion transmission was a two-stage gear train with a 1:16 gear ratio. Acrylic laser cut gears were used for the final robot which were chosen over 3D printed PLA gears due to superior efficiency. The robot layout had five different options that all underwent varying development, however in the end the fourth robot version which prioritised a low centre of mass and a relatively small rotational inertia was chosen to complete development through to the final demonstration.

The software started as an edge-detector test, then was adapted to bang-bang, proportional control then finally proportional integral derivative (PID) control. The robot completed the course in 10.08 seconds, meeting all requirements and placing second in the competition behind a group with a 9.85 second time. The most challenging issues the robot faced was gear slipping on the shafts and brownouts, the latter of which was not entirely absolved. Comparison with other groups gave potential solutions of thicker gears, which would come at a cost of a less compact gear train, or using friction wheels to lower the required rubber bands and overall weight. The group had effective planning and teamwork and learned to change one variable at a time when prototyping and to record more test data at early stages of the project.

Introduction and Constraints

This project covers the design process of a line-following robot. The robot was tasked with completing a course within a 2.4 x 1.2 metre track. Completing the course consisted of following the line from the start of the course to the end. The course was a black surface with a white reflective line that did not intersect itself. The line was constrained to a width between 15 to 20 millimetres. The course had a maximum line length of 15 metres with a minimum line radius and orthogonal separation of 80 millimetres. The robot had to start from stationary. Material usage was limited to the parts list specified in the brief, such as using less than 100g of PLA and having PCB dimensions that were less than 100 millimetres. This included a limit of 5 sensors, four AA rechargeable NiMH batteries and four motors that were either Multicomp MM10 or Multicomp MM28 (University of Canterbury, 2026). This report aims to describe the electronic and mechanical process of creating a line following robot, evaluate the robot's performance, and reflect on the development process.

Design Evaluation

Initial Design

The initial PCB and motion transmission design was based on a combination of each group member's individual submissions. One group member's board was used as an initial template. Through collaborative brainstorming, 3 LEDs were added, one for power and two for general use. A simple initial sensor board design was also produced.

An initial motion transmission design was adopted from one group member. Although the MM28 motor selection, usage of gears with a ratio of 1:16 and a wheel radius of 50mm were kept unchanged from the initial design, many CAD iterations of the gearbox housing and robot layout were undertaken as the robot became more advanced, more reliable and faster.

Boards

For our board, the ATmega4808 microcontroller was chosen over the ATmega328. This is because, despite being more difficult to solder, the ATmega4808 provided pins for more possible functionality, and included a built-in crystal oscillator meaning that external circuitry for a clock was unnecessary. The board was laid out such that the sensor connectors were all conveniently placed near each other, as well as motor pin connectors on their corresponding side of the board (see Figure 1). However, a vertical orientation of the main PCB on the robot was not considered, making these connector locations no longer optimal. This could have slightly increased noise and resistance due to having to use longer wires for connections.

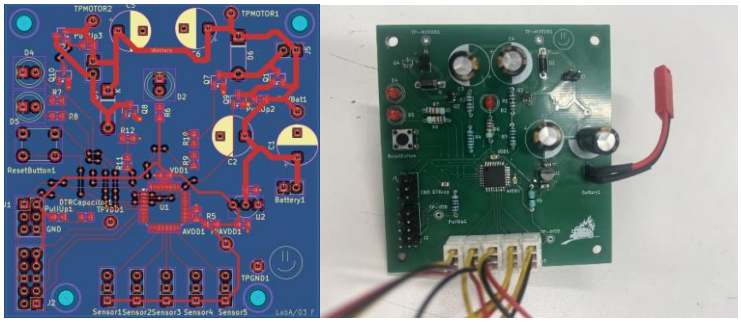


Figure 1: The final PCB design and board

A change made to the initial individual design of the PCB was the use of a PMOS to short the motor, using the back EMF to run the motor in reverse, which would effectively electrically brake the wheels. This PMOS was also driven by an NMOS, which could pull the PMOS gate to ground. However, this electrical braking was not used in the final design, and instead rubber bands were used for physical braking. This was because physical braking cannot damage the electrical components and is easier to implement. If the PMOS and the NMOS driving the motor driver were ever on at the same time, a short would be created, possibly melting our motor drivers. However, using physical braking came with drawbacks, as a higher PWM was required for the same speed, which caused brown-out problems. Browning-out refers to when large spikes of current are pulled by components such as motors so that an insufficient amount remains to power the microcontroller, causing the microcontroller to turn off and the robot to stop.

For each sensor board, a 3-pin connector was used, despite the TCRT5000 having four inputs. This is because one of the inputs is used to turn the sensor LED on. Due to no reason for this LED to ever be off, this input was wired directly to AVDD, sitting at 3.3V. Schmitt triggers were considered to digitise the signal before reaching the MCU, however the original sensor board design (Figure 2) was chosen as the time gained due to digitisation would be minimal and could likely cause noisier inputs due to more components. The original design works by using the TCRT5000 optical sensor as a simple pull down to ground when active, making the sensors read high while on black, and low when on white.

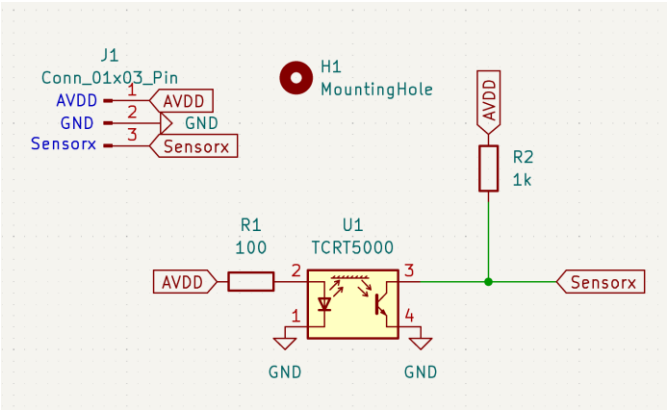


Figure 2: The final sensor schematic.

When testing the sensors, an appropriate sensor value needed to be chosen, to maximise the difference in output voltage between a black and white surface. To do this, current was read with a 1kΩ resistor on the board. As the current drawn by the sensor is approximately constant for a set light level. A measured current of 1.2mA, and a voltage drop of 3.3V can be used to calculate an optimal resistor value for R2. From the options available, a 2.2kΩ resistor gave us

the best result, with a black surface giving close to 3.3V, while the white line returned roughly 700mV (Figure 3). For data and code, see Appendices.

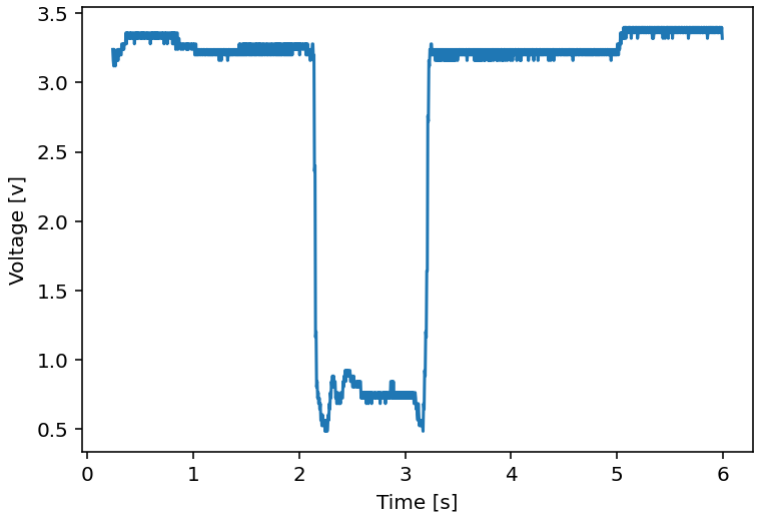


Figure 3: Sensor readings while crossing a white line.

During the testing process, browning out was a continuous problem. This was due to the implementation of rubber bands for physical braking, which meant a higher PWM must be used to achieve the same motor speed. This was solved by firstly replacing the wires from the battery to the main board with thicker wires, with better solder connections. An extra 100µF capacitor was also added to the battery connector, and both motor connectors, so that there was a total of 200µF at each of these locations.

Cad modelling and Motion Transmission

The MM28 motor was chosen over the MM10 motor due to providing more torque, as well as being able to be run at a higher battery voltage. It would be the more suitable option for a heavier robot; the robot would be heavier due to using gears instead of direct drive. Furthermore, since the motor had higher power, it indirectly meant it had better breaking power. This is because more rubber bands could be added to the drivetrain to increase the deceleration time as an effect of the tension of the bands without significantly impeding the top speed.

The chosen gear ratio was 1:16 reduction to slow down the motor's high rpm and increase torque, spread across two 1:4 gear stages. The maximum torque case would be acceleration from standstill, and calculations for this gave the lower bound of a gear ratio at 1:9. The upper bound was calculated from the low torque and high rpm case when hitting top speeds on a straight, this gave an upper bound of 1:20. The value of 16 was chosen due to its integer factorisation. (Appendix C)

Design Constraints for Motion Transmission and Robot Assembly:

- The robot must be easy to assemble. Assembly should take less than 20 minutes, and common mechanical troubleshooting and maintenance should take less than 10 minutes.
- The robot can only use material stated in the Line Following Robot Brief, found in the Tron lab or otherwise approved by tutors.
- The robots' mechanical system must be functional and reliably, it must be able to complete the track reliably (excluding external code or electrical errors) without breaking or otherwise preventing the robot from finishing the track.

Design considerations:

Want majority weight above (maximum traction preventing wheel slip during acceleration) the wheels and as low (maximum traction preventing wheel slip during acceleration) as possible. Keep weight distribution symmetric about both sides so that robot turns predictably. Need some weight on the front point of contact with the ground so there is enough downforce to keep consistent contact, however, do not want a lot of weight as this would increase friction.

The centre of mass should be low, for control around corners and to prevent forward tipping when accelerating. Due to these weight considerations, the robot's physical layout should be designed around its most dense components such as batteries and motors rather than the PCB.

Physical adjustability can be good; it can save a significant amount of time instead of doing CAD edits and waiting for a 3D print. However, in some cases, it could also take an unreasonable amount of time to make a part physically adjustable and would be better to create something permanent, even if arbitrary and then change the CAD design and 3D print the new version when iteration is needed.

A degree of modularity is good, having parts that can be assembled separately is beneficial for easy assembly and so that troubleshooting and maintenance is both possible and quick. Both too much modularity and adjustability, would lead to more points of physical wear and more connections that could fail.

Robot Body Designs:

All CAD modelling was done on SolidWorks 2023, with all 3D-printed parts printed in PLA with 0.4mm nozzle precision.

Gearbox Housing 1:

The method of creating the gearbox housing was to find a layout of the gears and then build a supporting part around them on SolidWorks. The layout idea would be to compact this linear gear system by making the second stage fold back on the first stage so it would take up less space.

Four spur gears were added into a new assembly, from toolbox add-in.

The gear dimensions were as follows (where m is module, n is number teeth and w is face width):

1. m = 1mm, n = 10, w = 3mm
2. m = 1mm, n = 40, w = 4.5mm
3. m = 1.125mm, n = 10, w = 3mm
4. m = 1.125mm, n = 40, w = 4mm

The process to create the gear spacing geometries did not involve any calculations. Instead, a sketch was made in the assembly and after moving the gears into alignment and mating them, the centre of the gears was sketched onto the part with convert entities feature. During this positioning, it was important to make sure the teeth would mesh, and there would be adequate distance between the intersections of the shafts and the supports side.

Shaft holes were given diameter of 7mm to fit a 6mm shaft and the motor shaft hole was given a diameter of 2.1mm to fit the 2mm shaft, the range of tolerances here was so that with the first print, multiple fits could be tested. The gearbox housing would be mounted onto the chassis using M4 screws.

Figures 4, 5 and 6 show the top view, isometric view and exploded view of the first model of the gearbox housing in assembly with the gears, shafts, bearings and a 3D-printed 50mm diameter wheel.

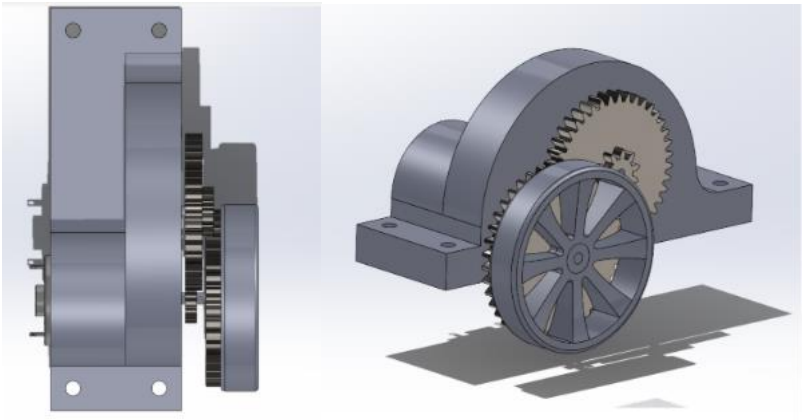


Figure 4, 5: Top view and isometric view of Gearbox Housing 1.

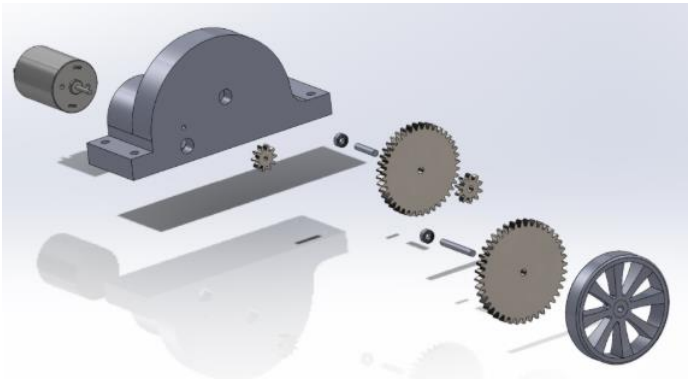


Figure 6: Exploded view of Gearbox Housing 1.

At this point in development, a potentially serious issue was identified. The two shafts connecting the larger gears and wheel would not penetrate the mounting block very far. It was likely that there would not be enough support due to the shallow holes having only one point of support with the bearing at the end. There was a possibility that it could be fine due to how short the shaft was and how lightweight the robot would be, however the ability to hold the shafts in around corners was not expected.

Figure 7 shows the first 3D print of the gearbox housing. The following was observed;

- The support was far larger and much sturdier than anticipated (no give felt or cracks heard/seen under compression and torsion by hand); so all of it could be made thinner and lighter.
- Motor fit was extremely tight; the motor could be squeezed in, but to take it out would require repeatedly hitting it with a screwdriver through the bearing hole on the other side. For a looser fit the motor holder diameter would be increased. The motor shaft hole could also be increased; this had no practical benefit other than ensuring complete clearance of the motor shaft and to remove more material.

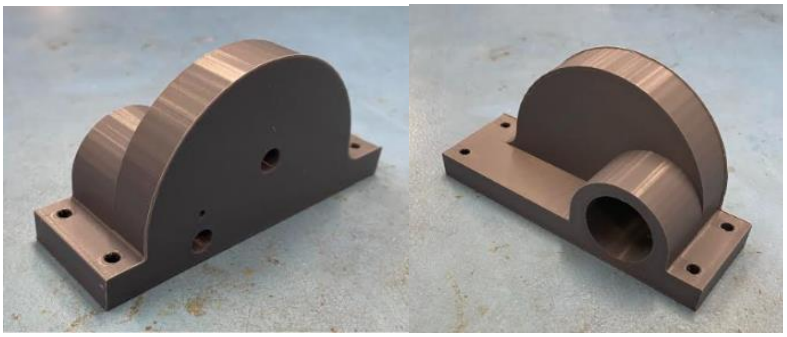


Figure 7: 3D-printed Gearbox Housing 1.

In figure 8, iteration 2 of the gearbox housing shows how the part was made thinner, this pattern of removing unnecessary material continued for the next couple of iterations. In iteration 2, a disassembly hole was added to aid in the removal of the motor. It would not be until iteration 4 that a gearbox housing part would be used in a full robot design. The diameter of the bearing holes would be decreased from 7mm to 6.2mm to house the 6mm bearings in a tight fit. In total 9 printed iterations were completed.

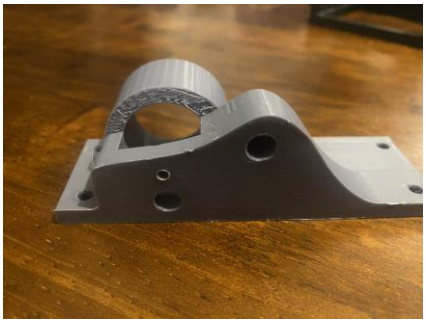


Figure 8: 3D-printed Gearbox Housing 2.

ROBOT 1: The Adjustable Robot

Due to wheel height and wheel spacing being unknown, it was decided to attempt a modular, adjustable design for the robot and its parts. This design was mainly undertaken with the intention of creating a prototype robot that could be tested and adjusted by hand to find adequate distances for these properties; however, its development never made it past test 3D prints to full assembly.

The robot's chassis would be a 3D-printed PCB holder, that the PCB and battery packs could slide in and out of. The PCB holder would have a large tab that the 3D-printed Motor Tab could slide up and down on, adjusting the height of the rest of the robot relative to the ground. The gear box would be attached to the motor holder and could slide inwards and outwards, changing the distance between the wheels. The gearbox housing would have its 4th iteration, the main difference to previous versions is the connection tab was moved to the top so the motion transmission could be placed underneath the Motor Tab. Figure 9 contains the CAD assembly of the PCB holder, the Motor Tab, an early sensor mount design and gearbox housing 4.

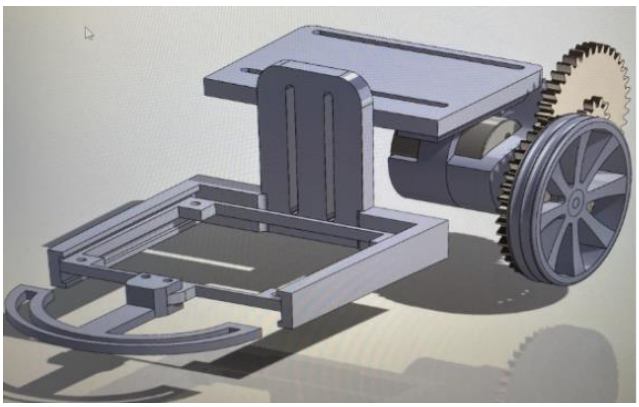


Figure 9: Isometric View of the Adjustable Robot without electronics.

A rolling castor ball was considered for the third point of contact with the ground but was discarded. The reasoning for this is having a rolling point of contact at the front could be inconsistent; if the ball slipped (changing from static friction to kinetic friction) around a corner or malfunction in other ways, this could cause unpredictable motion. A screw would instead be used for all future robots. This screw design is a good example of physical adjustability; the height could be adjusted manually with a nut and would be very unlikely to fail.

The adjustable robot would be deemed to be too time consuming for what it was worth when there were better alternatives to pursue. It would also fail the reliability constraint as the robot would have too many unreliable connections that could fail.

Robot 2: The First Prototype

Robot 2 was created quickly with the intention of using any prints we had so far to try and get a robot completing the track before any other group. It still used the PCB battery pack holder design but connected the motion transmission directly to its side, removing all the adjustable

tabs from the previous model. Minimal physical design considerations were adhered to, rather it was important to just produce a whole robot as fast as possible so that the code, mechanical and electrical systems could be tested together. Because of this, the rotational inertia was poor with a very wide robot and the connection between the motion transmission and rest of the robot was unstable, as seen in figure 10.

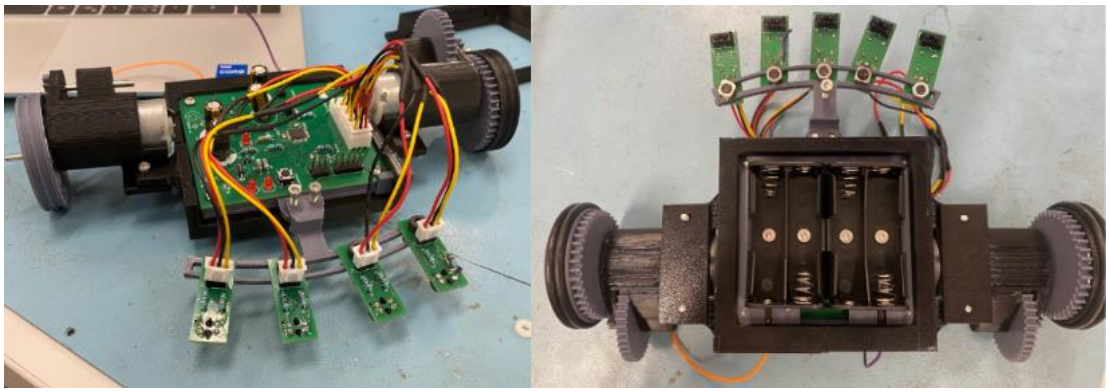


Figure 10: Photos of the first complete robot prototype.

This robot included the first full assembly of the motion transmission, using the 6th iteration of the gearbox housing part. As seen in figure 11, the key difference of this iteration was that it was extruded outwards with a cavity that allowed a gear to be set within; by making the shafts connected by two bearing at either end it could help keep the shaft stable and fixed within the housing.



Figure 11: Gearbox Housing 6.

Once assembled, the robot completed the course using bang-bang control, however when it came across two close parallel lines joined by a hairpin corner, the simple bang-bang control would make it join onto the wrong line each time. It was identified that the robot's turning radius was too wide making the sensors cross the line it had already come from. Code would need to be changed to prevent this, and the turning radius would need to be made smaller by making the wheel-to-wheel distance smaller and by adding braking on the inside wheel.

Robot 3: Minimising Rotational Inertia

The main intention for the third robot design was to see how small of an area the heavy components could take up, while also not stacking the batteries and motors on top of each

other. The aim was to make the footprint of this model smaller than any other in the cohort; we had an advantage over the other gear systems with how short and thin our motion transmission was. By bringing all the heavy components as close together as possible, the mass product of inertia would be minimised. The applications of this are it would allow the robot to rotate more quickly, due to less inertial resistance to rotation. Multiple layouts were sketched on paper and discussed.

Robot 3 used the 7th iteration of the gearbox housing part. This version contained cavities from overlapping adjacent chassis parts and batteries to allow a smaller fit.

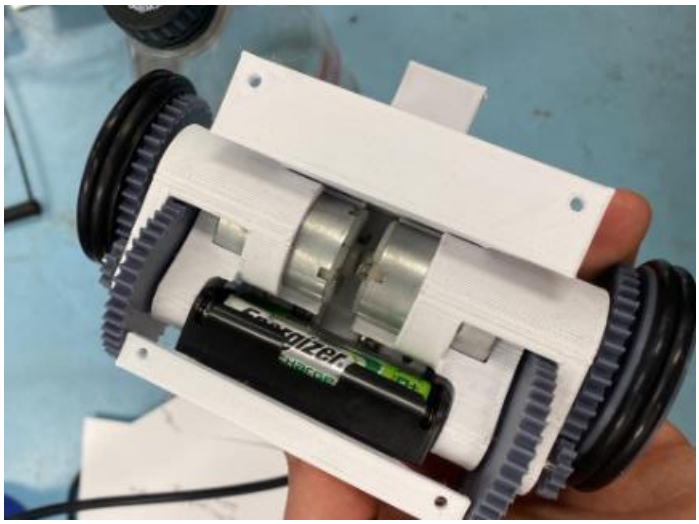


Figure 12: Chassis of Robot 3 including complete gearbox and vertically orientated batteries.

Robot 4: The Sailboat

As demonstrated in robot 3, there was a trade-off between how small of an area the robot could take up and how high the centre of mass would get from stacking and orientating components upwards. Due to having clearance under Robot 3, adding a dense flat weight on the underside of it was briefly considered. However, this would not move the centre of mass as far as just shifting the motors and batteries down. The goals for Robot 4 would be to improve Robot 3 by lowering the centre of mass as much as possible. This was done by lowering the batteries, as well as rotating them flat and by lowering the motors.

At this stage the height of the motors was fixed relative to the height of the centre of the wheel; to lower the motors the gearbox housing would need to be updated again. It was observed that rotating the motion transmission would change the perceived relative height of the bottom of the motor and the bottom of the wheel; this is because the wheel was not on the same axis as the motor. The idea came to make the gearbox housing attach to the chassis in a way that it could rotate, therefore tedious measurements and calculations would not need to be made, and the wheels could instead be adjusted to the height of the rest of the robot. If this was implemented then by rotating the gearbox housing, one could adjust the height of the chassis relative to the ground without doing any CAD adjustment and further 3D printing. This would save time and material resources if successful. This could be especially helpful if other wheel

sizes were to be tested, as rotating to compensate for changes of wheel height could keep the centre of gravity low and have ground clearance.

Note that the wheel size would be constrained to being larger than the gear on its axis, otherwise the gear would roll on the ground instead of the wheel. The size of this gear could be changed by adjusting the module and number of gear teeth. However, it would be important to keep the gear spacing the same, otherwise the part housing the gear train would not fit. The overall gear ratio would also have to be conserved. Since the robot had a two-stage gear system, to change the size of the 4th gear teeth could be swapped with the third gear and the first stage ratio could be adjusted to compensate for any changes in the second stage. This process could have a fast turn-around, much faster and easier than making large changes on CAD, because the gears could be made in a gear generator and manufactured quickly by laser cutting.

There were a number of requirements for rotating gear box to be successful; it must have many positions it can be changed to for it to be useful, it must be able to be locked into its position once set so that the robot is sturdy and the motion transmission will not move around or fall out, and it must ensure that the motion transmission parts do not get in the way of the rest of the robot when rotated. Other cylindrical mechanisms were considered, including those used in locks, and an axial tooth design between the contacts of the cage and the gearbox housing cylinder as shown in figure 12. The axial tooth design was abandoned due to complexity of modelling and not being able to prevent the motion transmission from falling out, despite having many positions it could rotate to and lock at.

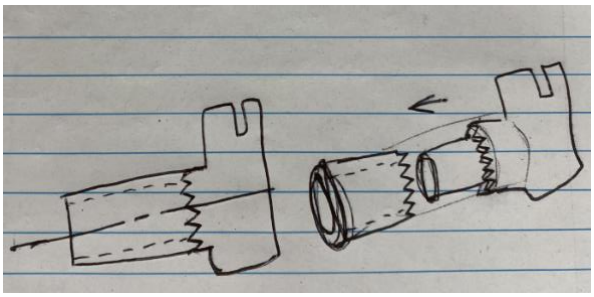


Figure 13: Abandoned axial tooth attachment concept, with inner cylinder gearbox housing attachment on the far-left sliding into the motor cage.

The idea settled on was laying holes for screws around the gearbox housing cylinder and the motor cage. To get more possible position to lock in, there would need to be some misalignment between the inner holes and the outer holes. Optimising this can turn into a number theory and combination problem, however the mathematical analysis occurred only after designing. At the time, it was recognised that the difference of spacings between the holes on the inner cylinder and outer cylinder should not share common factors, to prevent redundant repeating combinations, therefore spacing differences that were small to preserve a large number of holes but also did not share common factors would be most applicable. Coprime values of 3 and 4 would therefore be suitable for the misalignment between the inside and outside hole spacings. The holes in the inner gearbox housing cylinder were spaced 20° around as this was a suitable spacing for the hole size of 3mm, while the cage would have 17° spaced holes as well as 24° spaced holes, as seen in figure 17, applying a misalignment of 3° one way and differences of 4° the other way.

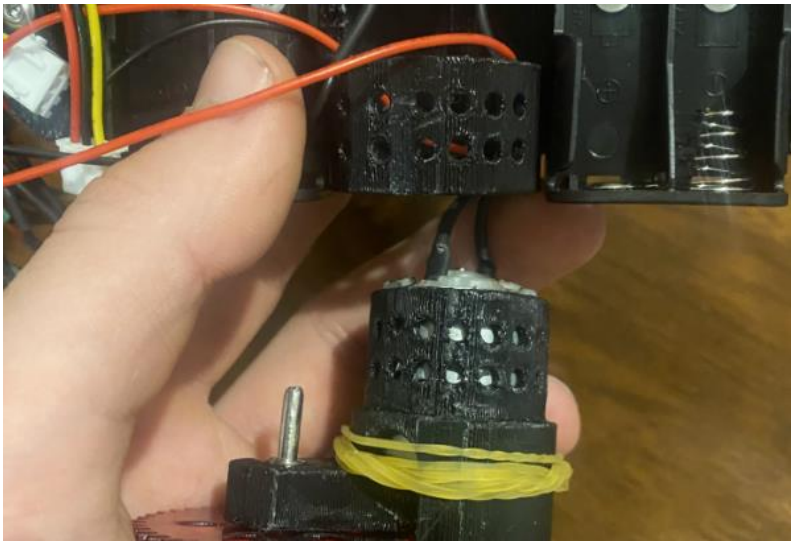


Figure 15: Outer cylinder (motor cage) above with 24° hole spacing to the left and 17° spacing to the right, and inner cylinder (gearbox housing) underneath with 20° spacing.

The application of differing hole spacings on the same ring on the outer cylinder this ended up creating irregular spacing between the positions that the attachment could be locked at, these repeated and clustered around every 20°. In retrospect, it would have been more optimal to have a different set of rings for both the 17° spacing and the 24° spacing, this would create more alignment positions and make more regular spacing.

While it was known the rotation would affect both the height of the centre of mass and its horizontal position relative to the wheels, it was not discovered until the mathematical analysis that it could be manipulated so that either the vertical or the horizontal displacement could be dominant, therefore targeting one orientational aspect over the other. The initial intention and use were only for the height; however, this discovery could lead to further refined development for more intentional adjustments.

In the end, the set up used created a cycle of irregular steps with a minimum angle step of 1° and a maximum of 11°. These corresponded to 54 positions in the 180° usable rotation before another gear would touch the ground, much more than the 9 possible positions if the spaces in the inner cylinder matched the outer cylinder. The arm from the centre of rotation to the centre of the wheel is 9.09mm. Rotation will displace the wheel on a circular path with this radius. The attachment will therefore allow both a vertical range and horizontal range of 9.09 mm to –9.09 mm.

This system which was initially a lazy shortcut so that the designer would not need to measure and calculate changes in height or rotate and print the part to the correct angle ended up being useful and interesting and although the applications were not fully explored, more intentional adjustment would be possible in further developments.

The outer cylinder of the rotating attachment deemed the motor cage would be fused with the chassis, as seen in figure 16. The rest of the chassis would have to be strong, lightweight and built around the heavy components to minimise the height of the centre of mass. The chassis would be built over the top of the batteries and contain the motor cages. Arches were used to provide tension to hold the batteries without bending of the chassis. The PCB would be

attached perpendicular to the ground on the top of the chassis, the purpose for this would be so that it is out of the way from the rotating motion transmission. As the PCB is a very light component relative to the rest of the robot, having it vertically attached instead of horizontally would have minimal effect on the centre of mass. Having it aligned along the axial direction of the robot would be best for rotational inertia when turning.

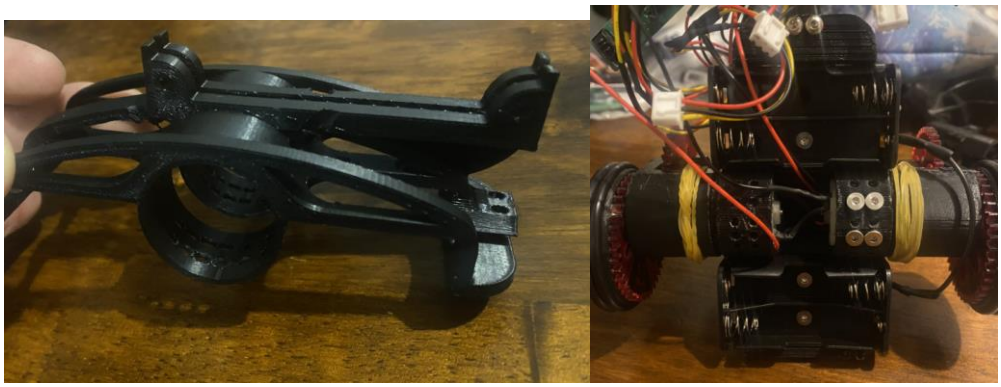


Figure 16: Photos of chassis highlighting frame design and location of motors and batteries.

A few issues were encountered with robot 4, the first being the motors were too close together meaning there was not enough space to attach the wires and they would often break. This was worked around at the time by only attaching the gearbox housing to the outer hole with one screw, this was adequate but made the robot less rigid as this connection allowed for some limited, undesired pivoting. Another issue was that the wires needed more space to get to the PCB, so holes were drilled through the top of the chassis. The final problem was that the wheel shaft could fall free from the gearbox housing, the solution to this would be to file a bur on one end of the shaft so that it would be held between the motor and bearing.

Despite these issues, the group was quite happy with this robot as it completed the practice track in 20.21 seconds with bang-bang control and no braking mechanism (motors running at 25% datasheet capacity). It also looked cool in all black.

With tuning, the robot was able to consistently hit 15 seconds. However, after a lot of trials, it eventually would get slower. The cause of this is hypothesised to be rubber bands losing elasticity, gears wearing down over time and slipping on the shafts when hot glue fails. It was observed that the charge on the battery would also greatly affect the performance.

Robot 5:

At this point in time, many other robots caught up in their development, some even significantly beating robot 4. The fastest robots tended to be far more lightweight and had their weight more concentrated over the wheel-to-wheel axis than robot 4. Because progress had slowed down, it was decided to make a robot with less inertia and mass. Robot 5 was made with design elements from 4 and 3; from 3, the batteries were kept closer to the wheel-to-wheel axis and from 4 the rotating motor transmission, figure 17. The chassis was made to minimise as much PLA as possible, figure 18; using the battery packs to provide most of the structure was considered however this would limit the positions they could be orientated in and since they were a dominant mass of the robot they were mounted onto PLA instead. Battery packs would be tied on with zip ties as nut access to the screws was not possible, this proved to be a very

simple and sound solution. The PCB was tied with fishing wire on top. The attachment of the PCB with fishing wire would take a long time to tie back on if the batteries needed to be replaced, a better solution would be to use velcro.

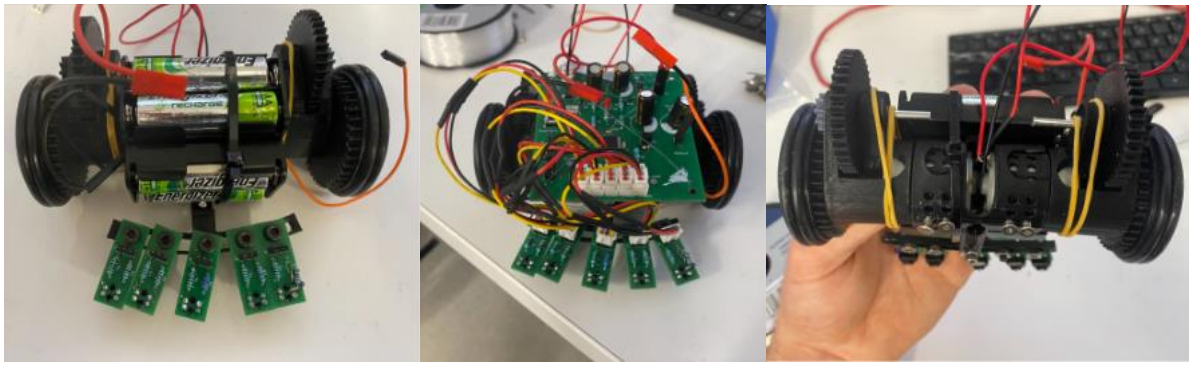


Figure 17: Photos of abandoned robot 5 design.



Figure 18: Chassis of robot 5.

This robot seemed to have more superior turning speed; however, the relatively high centre of mass that also was not far over the front of the wheels would cause it to wheelie. This design was then abandoned, because tuning such a different robot would take too much time. Furthermore, to prevent it from tipping a significant amount of weight would need to be added in front of the robot, this would defeat the purpose of this design by making it heavier than robot 4 and have worse inertia.

Robot 4 was chosen as the final design, and the group considered other factors limiting its performance. It was important to prioritise which limitations were the primary bottlenecks of performance, and which were more "glass-ceiling" limitations that were unlikely to be reached.

The gear transmission was identified as a limitation of priority. This is because poor meshing would greatly limit efficiency and gear slipping as well as gears catching on nearby surfaces could cause unpredictable movements limiting control.

Gears made from acrylic would theoretically be superior to those made from PLA, this is because they are a hard material that does not stretch so less energy would be lost due to elastic collisions between teeth, and because the tooth geometries could be manufactured more accurately with a laser cuter than the 0.4mm printers. 3D-printed gear teeth could also deform as they cool.

A 3mm sheet of acrylic was selected, but testing was required to get the right kerf tolerances that allowed tooth meshing. The acrylic gears meshed well so the gear box was assembled with them instead of 3D printed gears. The gears' hole was too big for the shaft; however, it could be filled using a bit of hot glue. While the first stage of the laser acrylic gears meshed well and lined up with the same distances, the second did not because the module was wrong, so a PLA gear was used as the final gear which worked fine.

With these gears we got a new best time of 11.9 seconds, however design mistakes adding with compounding mechanical wear was causing the robot to fall apart so often that testing code could not be done.

The following design issues plaguing the robot:

- Acrylic gear inner hole too big and hot glue not strong enough leading to gear coming free from shaft.
- Motion transmission not spinning well due to gear friction with nearby plane surfaces and bearings being damaged from assembly.
- Gearbox housing not connected to the chassis well due to the chassis not having sufficient space for motors and wires. This meant the gearbox housing would be connected only along one ring rather than two.

To fix the gear slippage on the shaft multiple ideas were considered;

1. Putting the gears onto a bearing on the shaft:
   Cons: need to redesign the gear box, which would take a long time to measure and test tolerances, and might not work.
2. Continued maintenance of the gears with hot glue when they break:
   Pros: do not have to redesign anything
   Cons: very temporary, will not make the robot perform better against or design considerations and constraints.
3. Fixing the gear onto the shaft before putting them in the gear box:
   Pros: would ideally only have to do once, hopefully no maintenance. Only a little change on the cad.

The third option of attaching the gear onto the shaft first was chosen. It was not known whether superglue would be strong enough to hold the acrylic on the shaft under torque. Superglue creates the strongest bonds between two surfaces when they are very close together, so the gears would have to be laser-cut again with smaller holes. A press fit would not work as the acrylic is hard and brittle and would crack if forced onto a shaft larger than the hole. Superglue is known to suddenly and catastrophically crack under torsion, so the motion transmission would have to be designed for easy maintenance. Some gear shafts were made with other methods, including roughing up the aluminium with a file and hole punching a small crater ring such that the elevated metal surface would lock onto the gear and hold it when put on, these were numbered so that if they were used then we could have a small sample of what methods work better. Many gear shafts were made to replace any breaks, figure 19, however during all later testing and final demonstration no gear shafts ever broke.

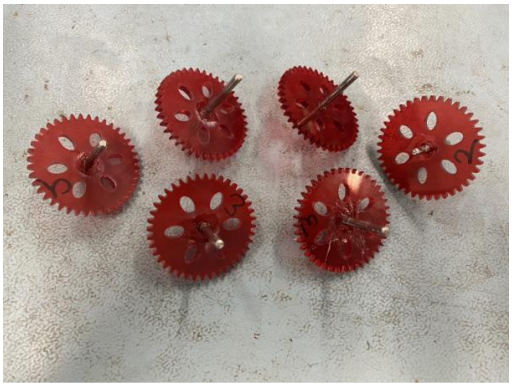


Figure 19: Spare acrylic gear shafts.

The gear case was edited such that the gear shaft could be swapped in and out as one part. This was done by cutting a path from the outside through to the shaft cavity, figure 20. During assembly, the gear shaft would be clicked into place, and then bearings would be placed over each end and into their cavities to hold the shaft in place. This would be the final change to the gearbox housing and motion transmission.

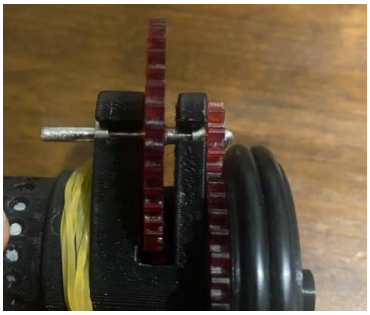


Figure 20: Shaft path visible down to bearings in final iteration of gearbox housing.

Wheels and Tires:

While wheel size was chosen arbitrarily and was not explored, tire selection was identified as an important factor for robot performance. The robot's control and speed around corners would be greatly impacted by the friction force experienced by the surface of the wheel. As friction force is equal to weight multiplied by the friction coefficient, increasing the friction coefficient would increase the friction force while keeping the same mass. Therefore, tires made from different materials were developed.

O-rings:

The original wheel used O-ring tires, these provided enough grip so were used for the final demonstration. The wheel had grooves of 3.6mm diameter inlaid on the outer radius for 3.5mm diameter O-rings. The depth of the grooves from the outer radius was chosen to be less than the radius of the O-ring, so that if the O-rings are sanded down so that their cross section is a half-circle for maximum friction, then the rubber will still be elevated from the PLA. The o-rings were never sanded down.

Bike tire:

The smooth inside of an inner bike tire was used for tire during robot testing. The ring would be cut from the tire, inverted so that the smooth ridge-less inner surface would contact the ground and would be stretched over a smooth-rimmed wheel. It provided a lot of grip, seemingly more than the o-rings, but it was decided to not include them as the robot was recording fast times with the o-rings anyway. According to online forums, in competitive remote control car racing, rubber tires can be soaked in WDF40 overnight to soften the rubber to get more grip. This was never tested.

Bathroom sealant:

Previous winners of the line following robot competition had made custom tires out of silicon. For this application, making tires from the same silicon resin would be too costly, take too long, and require both a vacuum and knowledge we did not have, so instead it was attempted with bathroom silicon sealant. The potentially useable set made was not tested because it felt like it had less grip than the bike tires which were working well, furthermore there were other more limiting factors for the robot's performance other than grip. (The molding process and results are contained within Appendix C iii.)

Laser Cutting

To improve the way the gears meshed for transmission efficiency, laser-cut gears were trialled (Figure 21). To do this, the gear models were converted to DXF files, then efficiently laid out on the Lightburn workspace. Due to the laser cut kerf changing the fit of the gears, different gear width adjustments for the best gear tooth meshing and fit to the shafts were tested. The best result was to increase the gear outer width by 0.125 mm and reduce the shaft hole diameter by 0.18 mm. After these adjustments were made, the speed ratio was set to 15/60 and the machine was set up and cut the gears. The gears were cut out of 3mm thick red acrylic. These gears, after adjustments to account for the kerf, meshed together cleaner then the initial 3D printed gears.



Figure 21: Acrylic laser cutting process.

There were some issues with the gears for this design. This was due to the gearbox being compact, so gears were this and had a small contact area with the shaft. This caused issues of

slipping. Having a tighter press fit would reduce this slipping, but could cause cracking as acrylic is brittle. PLA gears are more robust and crack proof but mesh less quickly, wasting more motor power as friction. Despite being brittle, as acrylic gears meshed better, they were more efficient and provided faster speeds. Therefore, acrylic gears were used In the final design.

Software and Code

The robot is controlled by Arduino IDE code (see Appendix D). The robot starts with a 1 second delay to give the group member providing power time to step away before the robot begun its motion. The code ran a loop every millisecond and read the five line sensor inputs using analogRead(). As the recorded reading of the sensors over the black ground was around 1015, the BLACK_LEVEL variable was set a little below this, at a value of 900. This black level was subtracted off the inputted reading for each sensor, as shown in the code below, so it could work out how much of the "line" each sensor saw.

```
int r[5];
for (int i = 0; i < 5; i++) {
  r[i] = BLACK_LEVEL - analogRead(sensorPins[i]);
  if (r[i] < 0) r[i] = 0;
}
```

Figure 22: Black Level Subtraction Code.

Each sensor was weighted by its position, with values of -1, -0.5, 0, 0.5 and 1 from left to right. The calculated error was a weighted average of these values, giving a zero value when the line is in the centre, a positive value if the line is to the right and a negative value if the line is to the left.

The error was used to correct the robot using PID control, as demonstrated in the code snippets below. The proportional term with a constant Kp = 30 provided correction based on how far away the line was from the centre. The derivative term with a constant Kd = 0.1 damps the oscillations by correcting based on how fast the error changed. As the derivative is divided by the 1 millisecond loop time, effectively multiplying readings by 1000, Kd needed to be small. A value of 0.5 was used earlier but was too strong and caused shaky unreliable movements and so was reduced to 0.1, which fixed this issue. The integral term with a constant Ki was to correct a constant offset which was not seen, Ki was set to zero.

```
error = weighted / total;

integral += error * DT;
integral = constrain(integral, -5, 5);
float derivative = (error - lastError) / DT;
lastError = error;

float correction = Kp * error + Ki * integral + Kd * derivative;

left  = baseSpeed + correction;
right = baseSpeed - correction;
```

Figure 23: PID Control Code.

This correction was used to accelerate one wheel while slowing the other to a base speed. These wheel speeds were sent to the motors using analogWrite(). If no sensor was seeing a line, the robot moved toward the side that the line was seen on most recently to ensure that the robot did not get lost at sharp corners.

The code was implemented in a way that complimented the wheelbase of our robot. As the wheelbase was similar to the minimum corner radius of the track, it allowed us to lock a wheel in place and turn the outside wheel until the line was found again. Therefore, the code was designed such that the system acted as PID on straight lines, keeping the line in range of the sensors at all times. Whenever the line was lost, the inner motor would stop, and the outer motor would drive the outer wheel at TURN_SPEED towards the side where the line was last seen until a sensor found the line again, like bang-bang control. This strategy worked for sharp corners and the robot switched back to PID control on wider corners and straights.

As the motors were started at stall, they had a high starting ramp up speed that drew a large current. On the day of our project, one of the motors had an issue and needed replacement. The large current caused a temporary drop in voltage of the motors, causing them to brownout, which meant they stalled. The rubber band braking also added to this as it meant the robot needed to run at a higher pulse width modulation (PWM). To reduce this on the day, the motor was replaced and the software was adjusted to start at a PWM of 130 then ramp up to a maximum speed of 212 and settle for a straight speed of 170.

Initially, the software was developed to implement an edge detection test to see if the sensors and motors worked as planned. The code was then adjusted to implement bang-bang control, just using just sensor two and four, where each motor was either completely on or completely off, depending on what sensor saw the line. This enabled the robot to move along the line and complete the track, but the movement was jerky and slow. The code then was edited to implement P control first for steering correction with all five sensors and then D control to lessen oscillations. This PID control was then further fine-tuned for optimal speed around the track. These updates and changes are shown in Table 1 below. On the day of assessment, the code was further fine-tuned by changing the constant values at the top of the code to provide the optimal run times. This proved to be effective, due to the robot gaining the second fastest runtime in the cohort of 10.08 seconds around the track.

Robot Performance

Throughout the project, the track times were slowly brought down through changes to code and mechanical design, up to the final run, in which a final time of 10.08s was achieved (Figure 7), giving us 2nd place.



Figure 24: 10.08s time in demonstration.

On the second official run, a time of 15.10s was achieved. However, in this run the robot browned out and reset, causing the 5 second delay at the start of the code to trigger again. Accounting for acceleration time and brown out time, this run would have been under 10 seconds and potentially a winning time without brown out. This shows how important ironing out issues like brown out are, and is something to focus on in later projects.

It was found that once code reached a certain level, it was the mechanical design that would bottleneck speed, and the design must be changed in order to achieve faster times. The only way to figure out wether or not the mechanical design was bottlenecking speed was to do thorough track testing with code, before making changes to the mechanical design and comparing new times to the old times (Figure 8). The biggest factor that limited the robot performace was braking around corners. As the motor drivers did not have an H-bridge design, the motors could only run forwards, so other methods must be explored in order to slow down the wheels on corners. Whenever rubber bands were added a significant improvement in speed was observed, due to being able to traverse corners quicker.

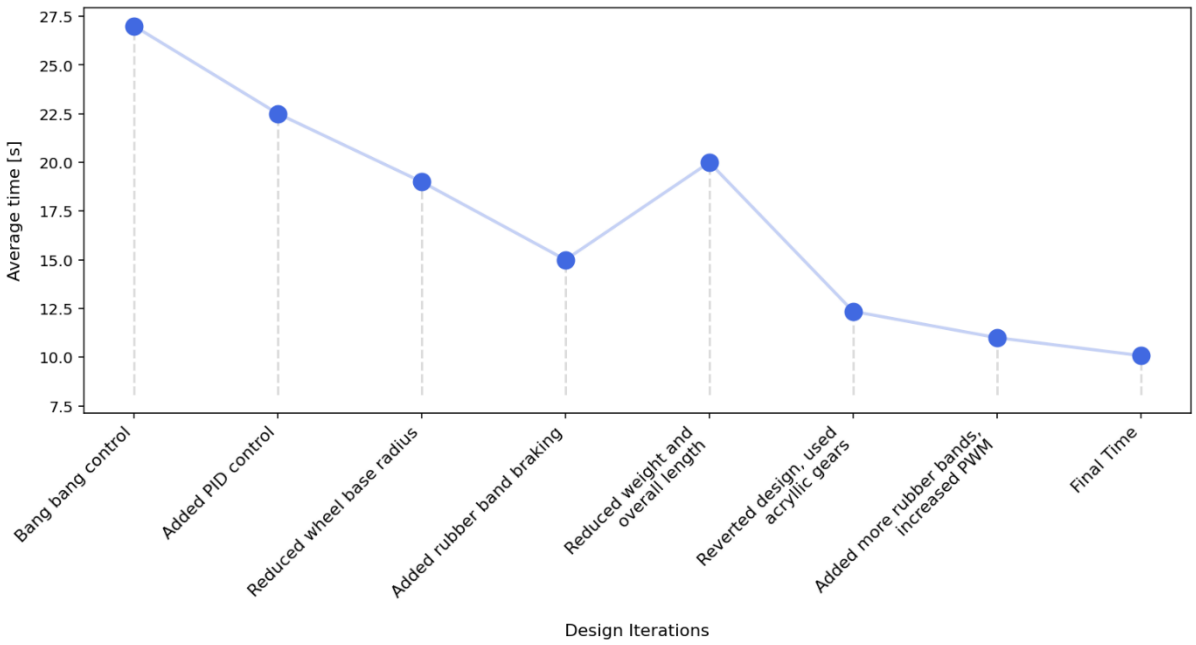


Figure 25: Average Time of Robot VS Changes Made.

Evaluation Against Requirements

Our project required that the robot that was able to successfully complete the provided course. Additional personal requirements were added, including having the robot complete this course in under 15 seconds. Our aim was met as the robot was able to complete the course in 10.08 seconds, giving us a placing of second overall. Our group also grew in our understanding of KiCad, CAD modelling, laser cutting, 3D printing, soldering and coding using Arduino IDE. The requirements of having no more than five sensors, no more than four AA NiMH cells and an allowed MM28 motor were also met. Only the required electronic components were used, the robot was under the limit of 100g of PLA, and the PCB dimensions were under 100 millimetres. The robot could also start from stationary

However, our robot fell short in its reliability. On the day of running, brownouts caused mostly by the higher PWM and the thin acrylic gears slipping caused the runs on the track to be less consistently successful. In future, we could make changes to improve our robot, such as remodelling it to require a lower PWM, using thicker gears, and possibly using a flyback diode to help reduce brown out issues. When choosing our gear ratios, a 1:16 ratio was chosen, which was in the middle of the calculated range, causing a balance of efficient straight speed and cornering ability. However, as the cornering was efficient and our robot had sufficient torque, the gear ratio could have been dropped slightly, to a ratio such as 1:12. This ratio would have had less, but still sufficient torque but would have had a faster straight speed.

Unique features

Our robot was unique in the way that it used a vertical PCB placement, and using a fin and decals to create a shark look. Placing the PCB flat would cause spacing issues, and due to its relative light weight, we could opt for a vertical board. Due to the way PID creates a weaving motion down the line, the robot was designed to look like a shark swimming through water.

The robot had red laser cut gears which provided large efficiency gains compared to PLA gears, which allowed more consistency between motor speeds, allowing us to have more control over how much friction we could create with the rubber bands, while keeping the motor speeds similar. We found that when we tried adding rubber bands with PLA gears, the motors would run at different speeds and sometimes get stuck. Throughout the design process, the chassis was designed to create as low mass distribution as possible, with the batteries and motors sitting less than 10mm off the ground.

Our group was also unique in the way that we started prototyping for our project very early on, enabling more time for refinement and creativity, especially in the chassis design.

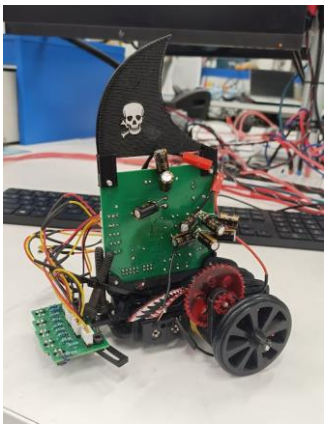


Figure 26: The Robot on the Day

Comparison with Other Groups

Throughout the design and testing process, there were two groups in particular that were used to compare ideas and evaluate the pros and cons of our design choices. These two groups were 01 / C, and 02 / G.

01 / C:

Group 01 / C ended up coming first, with a time of 9.85s, while we came second with 10.08s. This made the comparison of our design choices particularly interesting, as despite both competing at the top, both designs were quite different. Their group used direct drive, and had a much smaller wheel radius than us, and was lighter, weighing 299g compared to our design at 375. Using direct drive allowed for quicker response, due to no backlash that was caused from our two-stage gear system. The smaller wheelbase also allowed for a tighter turning radius, which could allow quicker turns. However, our larger wheelbase worked well for our design, as we could lock and pivot around one wheel due to having a similar wheelbase as the radius of the corners. Due to the friction wheel, they did not need to many extra rubber bands for braking,

which meant they could run at a lower PWM, causing them less brownouts, which was a big problem for us on the day of the demonstration.

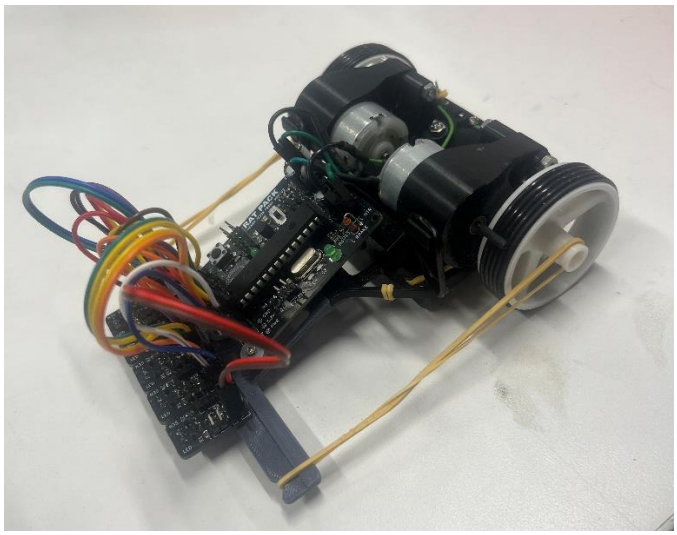


Figure 27 – Group 01 / C's Robot

02 / G:

Group 02 / G came fourth overall, with a time of 10.64s. The design itself was very round and flat, giving a low COM that would give less slippage around corners, greatly decreasing final time. The group also had a startup sequence every run, in which the sensors would be moved back and forth across the track line, and the average value of each sensor would be taken as the white sensor reading. This meant that if lighting changed the sensor values when the track was moved downstairs, their robot could easily adapt and run the track the same as before. However, the most important takeaway that we would incorporate from their design is the bevel gear system they implemented. One of our biggest problems was having thin PLA gears that would slip on their shafts, causing unpredictable behaviour on the track. This could have been fixed by using thicker gears, alongside a press fit. This group achieved this, while keeping the transmission system compact through the use of a bevel gear system, in which large contact area between gears was maintained, as well as thick gears for a stronger connection to their shafts.

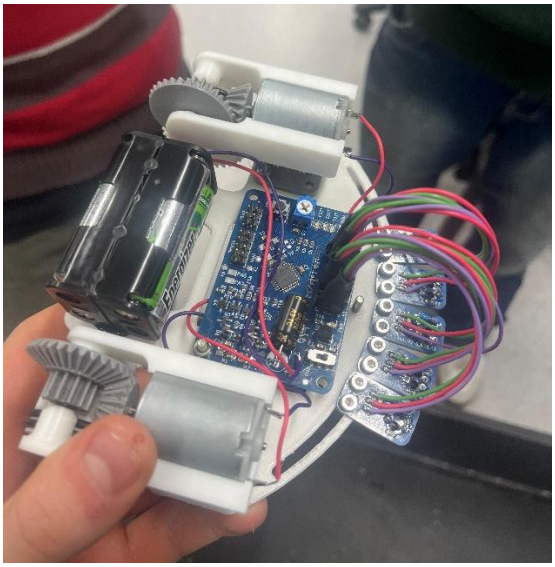


Figure 28 – Group 02 / G's Robot

Group Reflection

Project Process

The majority of our project process went as planned. Our group met well in advance before any briefing came out and decided to get a headstart in development by ordering the PCBs ourselves. This let us have a working PCB weeks ahead of many other groups. We were able to write basic bang-bang control code early on to get a robot that met all basic requirements and finished the track. The motion transmission did not go to plan initially, due to issues with the 3D printed gears. We switched to laser cut acrylic gears and had initial issues with laser cut kerf, but after adjustments, these gears meshed better. We fine-tuned the robot code and made many adjustments in the physical robot in order to get the fastest time possible. On the day, we had issues with a motor and brownouts, but we fixed these by replacing the motor and adjusting the code and were able to successfully have the second fastest robot in the competition.

Teamwork

Our group had a group chat and arranged to meet up regularly to work on the project. As the project progressed, we each took on separate roles but stayed together to ensure we integrated our progress. For example, at one instance in time, one member soldered the spare PCB, while one member made CAD adjustments to the chassis, while another trialled laser cut gear proportions and another member adjusted the code for more efficient motion. Having a testable robot early in the development meant that many separate tasks were able to be done synchronously, it meant that our group was able to progress through all required tasks fast and work to the individual strengths and preferences of each member. Our group members shared and discussed changes when relevant, meaning every member was aware of the current progress and no major disagreements arose. While pushing for faster times, some small disagreements came up regarding changing the design for small speed gains and weighing up the risk and rewards. This was resolved by ensuring design changes could be easily revertible,

so that the old design could be restored if a new change was unsuccessful, and by trusting a team members judgement.

Meta Learning

We gained practical skills in CAD modelling, laser cutting, 3D printing, circuit board design, Arduino coding and soldering. This project improved our group members time management and self-directivity as we learned to keep stages of our project completed ahead of time so that further improvement could be made. This made the primary drivers for motivation fun and eagerness to learn and improve rather than stress to complete it. One lesson we learned was to change one code variable at a time when tuning. Earlier on, changing multiple constants before running code made it difficult to know what impact each variable change had on the code. Testing each variable one by one allowed errors to be found faster too. We also learned to record more data from our runs in the future, as near the start of our process, we didn't keep track of all changes.

There is a distinction to be found when division of labour can be helpful in a group project and when it can be limiting. For something such as CAD modelling, it is best for the person who designed it and therefore understood it the most to make iterative edits. Splitting labour between domains such as CAD modelling, electrical troubleshooting and coding made sense as team members were able to specialise in one area and not have to explain every minor change and issue to a cooperating member, nor make mistakes by changing something in an area they did not properly understand.

It was observed that team performance, enthusiasm and motivation (all of which anecdotally correlate) increased by making the robot more unique to the team through adding personal touches. Examples of these design choices in our robot that would not directly affect performance included the consistent black scheme of the PLA parts, red colour and cut out styles of the acrylic gears, stickers, images on PCB silkscreen, vertical orientation of the PCB board and the shark fin. For future projects it is good for team moral to encourage all team members to contribute to seemingly less important aspects of the design, such as aesthetic choices. This makes the robot seem more of a possessive extension of team members and can cause more personal pride in one's work.

Conclusion

This project aimed to design, build and program a line following robot that could complete the given course in less than our goal of 15 seconds. The robot met the requirements of both the brief and our own personal target by completing the course in 10.08 seconds while staying within the required material specifications and other constraints. This time made the robot come second place in the cohort. This result was achieved by using as ATmega4808 board, five TCRT5000 sensors, effective motion transmission with laser cut gears, and fine-tuned software. We prototyped our robot early and were able to change and adapt it as the project continued. The robot had issues of gears slipping on shafts and brownouts causing less reliable runs on the track. Taking inspiration from other groups, these issues in our robot could be improved in the

future by thicker gears or friction wheels. Overall, our group gained valuable experience throughout all stages of the project and worked together well as a team.

References

University of Canterbury. (2026). *ENMT221 Individual and Team Line-Following Robot Project Brief*. Christchurch.

Gear Generator 2. *geargenerator.com*

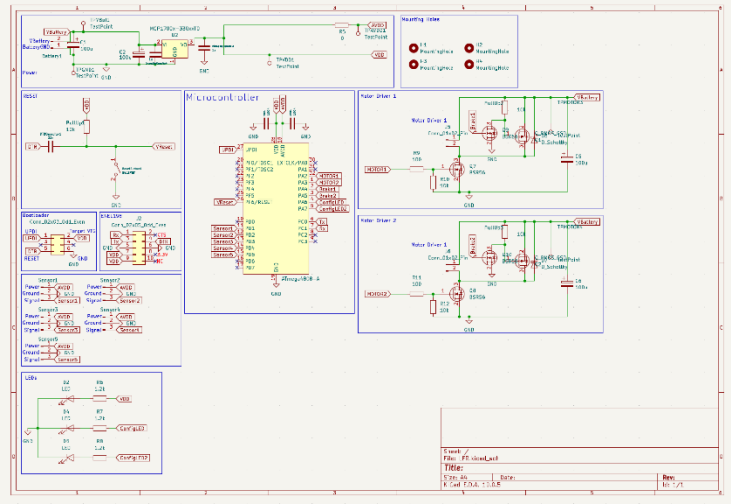
Free Gear Generator | DXF & SVG. *geardxf.com*

Thanks to all the tutors for their help. Also, thanks to second-year mechatronics peers Cacia and Peter as well as MathSoc associates Alex and Andrew for helping check the numerical / combinational calculations and theory for the rotating attachment.

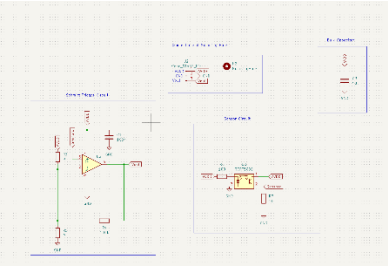
Appendices

Appendix A

Microcontroller Board Full Schematic

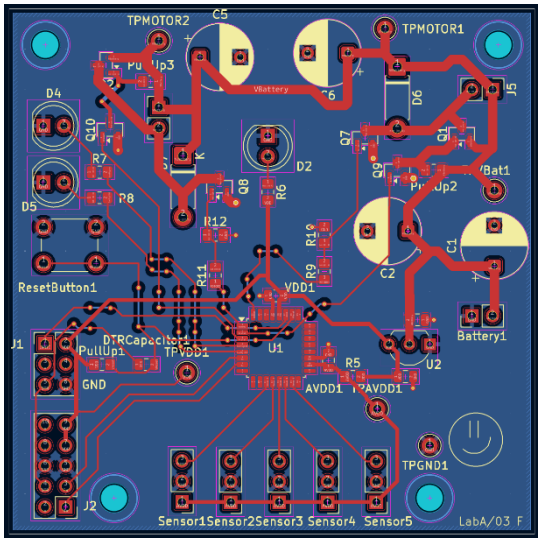


Sensor Board Schematic

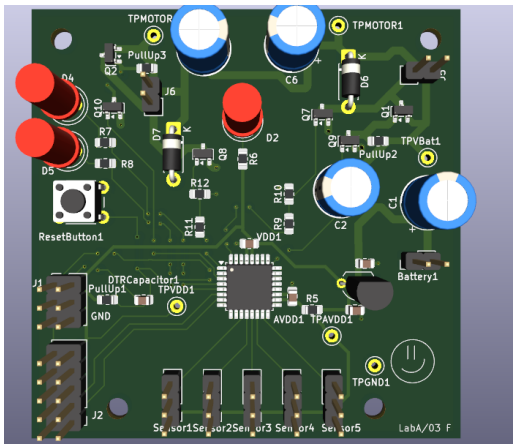


Appendix B

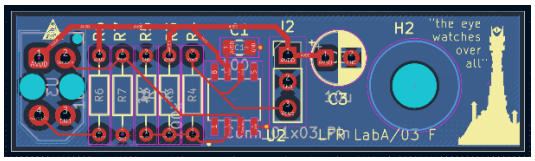
Microcontroller Board PCB Layout



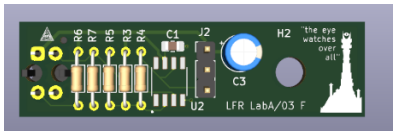
Microcontroller Board 3D View



Sensor Board PCB Layout



Sensor Board 3D View



Appendix C

i) Justification for chosen Gear Ratio:

Estimating the mass of the line following robot:

- 4 x AA batteries, 30g each
- PCB approximately 30g
- 3D printed body and wheels 204g
- 2 x MM28 motor, 42 g each
- Miscellaneous (bolts, fasteners, tires, glue...), estimate 30g

Total estimated mass = 4 x 30 + 30 + 204 + 2 x 42 + 30 = 468 grams

As shown in equation 1, the weight force on each wheel, F_w, was calculated to therefore be 2.30 N.

$F_w=(m*g)/2=(0.468*9.81)/2=2.30N$ *Fw=m∗g/2=0.468∗9.81/2=2.30N*

The maximum force for no-slip was then calculated, with the coefficient of static friction between rubber tires and painted ground assumed to be around 0.8.

$F_{max}= \mu_s*N= \mu_s*F_w=0.8*2.3=1.84N$ *Fmax= μs∗N= μs∗Fw=0.8∗2.3=1.84N*

Given the maximum force, the maximum applied torque could then be calculated, making the wheel arbitrarily 50mm in diameter.

$\tau_{max}=F_{max}*r_{wheel}=1.84*0.025=0.046Nm$ *τmax=Fmax∗rwheel=1.84∗0.025=0.046Nm*

Gear ratio for standing start:

From the MM28 data sheet, at 3V the stall torque is 122 g/cm which is larger than the maximum torque at 6V, 94 g/cm. It should be noted that at higher voltages the stall torque should be larger, this application will involve a maximum voltage of 4.8V supplied by the 4 x 1.2V batteries so the 3V stall torque should be treated as a lower bound when regarding acceleration from standstill. 122g/cm converts to 0.012 Nm.

For each spur gear stage, efficiency is around 70% for the 3D printed PLA. Efficiency is multiplied at each stage, so two stages of gears will give an efficiency of 49%.

$ratio=\tau_{max}\tau_{motor}* \mu_{gear\ system}=0.0460.012*0.49=7.8$ *ratio=τmaxτmotor∗ μgear system=0.0460.012∗0.49=7.8*

This gear ratio is for application of maximum torque, limited by tyre grip, and sets the lower limit of range of gear ratios suitable for the line following robot.

Gear ratio for straight line speed:

This ratio sets the upper limit of a suitable gear ratio; in the case of straight-line speed, high torque is not required. Instead, significant output power at the targeted speed is preferred. Intend to hit top-speed target at 80% of motor no load speed, because this is faster than the example in the brief and the team wanted to be competitive.

Motor no load speed = 9600 rpm

$\omega = 2\pi/60 * rpm = 2\pi/60 * 9600 = 1005 rad/s$ $\omega=2\pi/60*rpm=2\pi/60*9600=1005rad/s$

Target speed chosen to be 1 m/s, because top speeds over 1 are considered fast for non-competitive line following robots.

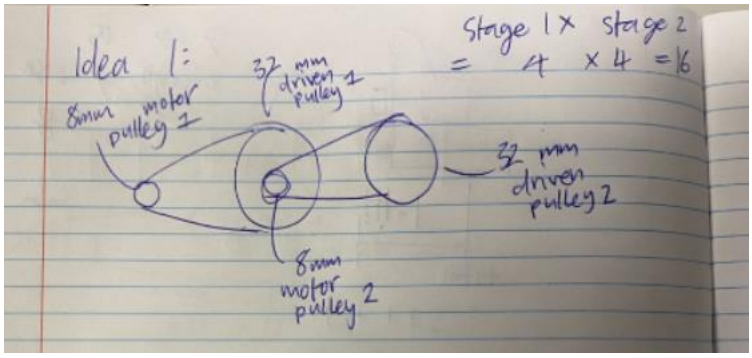
$ratio = 0.8 * motornoloadspeed * r_w / targetspeed = 0.8 * 1005 * 0.025 / 1 = 20.1$ $ratio=0.8*motornoloadspeed*rw/targetspeed=0.8*1005*0.025/1=20.1$

Therefore, a step-down gear ratio range of around 8-20 could be used in this project. The lower bound would not be the most suitable option because accelerating from standstill would have a small effect on the total time to do the course. Although a fast top speed is desirable, it was expected that there would be plenty of sharp corners in the track, so adequate torque would still be important for accelerating and decelerating fast enough. Therefore, a gear ratio closer to the middle value 14 would be a good starting point for development. A gear ratio of 16 would be nice to work with because of its integer factorization.

Chosen gear ratio = 1:16

Other forms of motion transmission were considered including belt and pulley as well as direct drive onto a friction wheel.

Belt and Pulley:



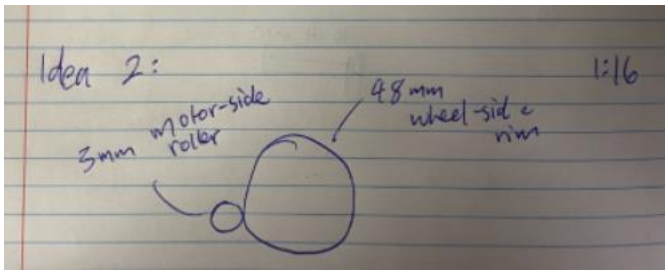
- Two 1:4 stages keep the total 1:16 ratio
- Efficiency assumption: belt/pulley losses come mainly from elastic slip under load rather than tooth friction. Assume 85% per stage: 0.85 x 0.85 = 0.72

- Resulting wheel torque (using 3V stall torque, 0.012Nm)

$\tau_{wheel} = 0.012 * 16 * 0.72 = 0.138 Nm$ $\tau wheel = 0.012 * 16 * 0.72 = 0.138 Nm$

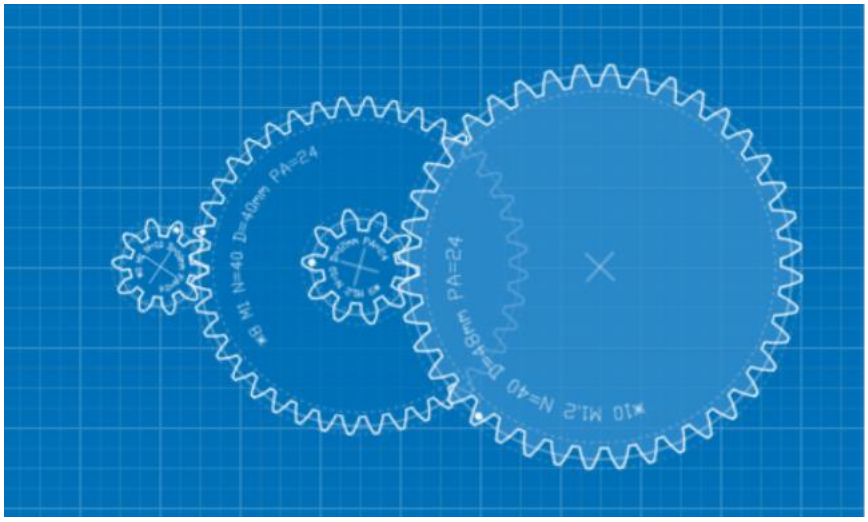
- This design needs a tensioning mechanism; it is also less precise and repeatable than gears. Elastic belts may lose elasticity from time under tension leading to decline in performance and replacement.

Friction Wheel Drive:



- 1 stage, 1:16 ratio
- Wheel torque highly variable, slip-limited

The website Gear Generator 2 was used to help visualize what a two-stage solution with gear ratio around 1:16 could look like. The following screenshot contains the linear gear system decided on; in the first stage a gear with module 1mm and 10 teeth is connected to a gear with module 1mm and 40 teeth, in the second stage a gear with module 1.2mm and 10 teeth is connected to the final gear with module 1.2mm and 40 teeth. The module was chosen so that the gears would stay reasonably small without risking tooth meshing problems that could arise from the 3D printer being unable to print smaller teeth accurately. The module on the second stage is larger because in the second stage there will be more torque and the teeth will need to be stronger as a result. Furthermore, this makes the final gear bigger than the driven gear of stage 1 which prevents the shaft in the final gear intersecting other gears if the second stage was to be folded back on the first stage.



ii) Mathematical analysis of Rotational Attachment

Rotating the inner cylinder to the next hole in relation to the 17° outer pitch would rotate 3°, and in relation to the 24° outer pitch would rotate 4°. The outer holes are spaced -48°, -24°, 0°, 17°, 34°, 51°. The inner cylinder repeats every 20°, therefore the mod 20° spacing is 12°, 16°, 0°, 17°, 14°, 11°. Measured from 0°, the lock positions are 0°, 3°, 4°, 6°, 8°, 19°, repeating every 20°. Therefore, the steps are 3°, 1°, 2°, 2°, 11°, 1° repeating. The finest step is 1° and the largest is 11°.

The arm from the centre of rotation to the centre of the wheel is 9.09mm. Rotation will displace the wheel on a circular path with this radius. The attachment will therefore allow both a vertical range and horizontal range of 9.09 mm to –9.09 mm. Each degree of rotation moves the wheel 0.159mm along its arc.

iii) Sealant Molding Process

A test was first done, proving that it would stick to PLA and could have a be soft and smooth while not breaking apart. Then molds were made. A wheel part would be placed inside a mold, which would consist of a solid base, a thin cylindrical surface for a smooth finish and generated support for the walls. The solid base would have holes at the bottom, annular holes to allow air to escape so that the silicon could fill it entirely and a center hole through which a shaft could hold the outer mold and inner wheel concentric. The solid base and thin cylinder were covered in vaseline so that the tire could be removed from the mold, but was not put on the wheel. It was fairly difficult to put the sealant in the cavity due to the thinness of the mold cavity and viscousity of the sealant, however a method was found to poke it around with the nib of a pencil and smooth it with the rubber end iteratively until it would come out the holes at the bottom. The molds were left to set for a day before being taken out.



This process had limited success; two sets of tires were made in case of a failure with one set turning out unusable due to too much incomplete casting. The surface was pretty smooth although had some air pockets and was soft yet strong enough that it would break apart from the wheel.

Appendix D

Full Code

```
// Code for our LFR with PID control.
// White line on a black surface. Sensors read LOW over the white line
// and HIGH (~1015) over black.

// Sensor pins with different weights. Sensor 1 (A1) has weight -1.
const uint8_t sensorPins[5] = {A1, A2, A3, A4, A5};
const float   weights[5]    = {-1, -0.5, 0, 0.5, 1};

// Left motor
const uint8_t MOTOR1 = PIN_PA2;
// Right motor
const uint8_t MOTOR2 = PIN_PA3;

// PID control values

// Kp constant. Error is at most +/-2, so the biggest correction is Kp * 2.
float Kp = 30;
// No integral control
float Ki = 0.0;
// Derivative is divided by a tiny DT, so it is a big number.
// 0.5 was far too strong. Try 0, 0.02, 0.05, 0.1.
float Kd = 0.1;


// Straight-line speed (analogWrite range is 0-255).
const float STRAIGHT_SPEED = 170;
// The maximum speed that will be sent to the motor during normal running.
const float MAX_INPUT_SPEED = 212;
// Lowest PWM that reliably turns the motor. Find it by testing.
const float MIN_SPEED = 0;
```

```
// Speed of the outer wheel when the line is lost (sharp corner recovery).
const float TURN_SPEED = 160;

// Start boost: accelerate to BOOST_SPEED, hold it, then drop to
STRAIGHT_SPEED.
// Speed at the very start of the ramp (must be enough to turn the motors).
const float START_SPEED = 130;
// Faster speed used during the openig boost (max 255).
const float BOOST_SPEED = 212;
// Time taken to ramp from START_SPEED up to BOOST_SPEED.
const unsigned long ACCEL_MS =700;
// Total time from the start that the boost lasts (includes the ramp).
// After this, speed drops to STRAIGHT_SPEED. Keep this >= ACCEL_MS.
const unsigned long BOOST_MS = 2400;
// When the run began (set at the end of setup).
unsigned long startTime = 0;

// Reading over black (HIGH). Measured ~1015, so use a value a bit below it.
const int BLACK_LEVEL = 900;
// With one sensor on the line, total is about 800. Over black it is ~0.
const int LOST_TOTAL = 200;

// The controller runs every LOOP_MS milliseconds. DT is the same time in
seconds.
const unsigned long LOOP_MS = 1;
const float DT = LOOP_MS / 1000.0;

// How many sensors either side of the last-seen sensor may see the line.
// 1 = the line can only move to an adjacent sensor each loop.
// (Does not apply to the outside sensors: see the window code in loop().)
const int WINDOW = 1;

// If the line is lost for this long, stop trusting the old position and
// look at all sensors again (e.g. after being picked up and put down).
const unsigned long REACQUIRE_MS = 500;

// Sensor index (0-4) the line was last seen on.
int lastIdx = 2;
// false = no valid position yet, so look at every sensor and pick the best.
bool tracking = false;
// When the line was last seen inside the window.
unsigned long lastSeenTime = 0;

// Memory between loops.
float integral = 0;
// Previous error.
float lastError = 0;
// Which side the line was last seen on: +1 = right, -1 = left.
```

```
// Starts at +1 so the robot has a direction if it begins off the line.
int lastSeenDir = 1;
// When the controller last ran.
unsigned long lastTime = 0;

void setup() {
  delay(1000);
  pinMode(MOTOR1, OUTPUT);
  pinMode(MOTOR2, OUTPUT);

  // Serial1 = USART1 (TX on PC0).
  Serial1.begin(9600);

  // Start the clocks now, after the 5 s delay.
  lastTime = millis();
  startTime = millis();
}

void loop() {
  // Only run once every LOOP_MS, so the timing stays steady.
  if (millis() - lastTime < LOOP_MS) {
    return;
  }
  lastTime += LOOP_MS;

  // Read all five sensors and work out how much "line" each one sees.
  // Read all sensors first.
  int r[5];
  for (int i = 0; i < 5; i++) {
    r[i] = BLACK_LEVEL - analogRead(sensorPins[i]);
    if (r[i] < 0) r[i] = 0;
  }

  // Decide which sensors we are allowed to look at.
  int lo = 0, hi = 4;                       // not tracking: all sensors
  if (tracking) {
    if (lastIdx == 0) {
      // Outside sensor (left) was triggered: the next sensor that can
      // trigger must be the one adjacent to it.
      lo = 1;
      hi = 1;
    } else if (lastIdx == 4) {
      // Outside sensor (right) was triggered: only the adjacent one counts.
      lo = 3;
      hi = 3;
    } else {
      lo = max(0, lastIdx - WINDOW);
      hi = min(4, lastIdx + WINDOW);
```

```
    }
  }

  // Only sensors inside the window count. Find the strongest one too.
  float total = 0;
  float weighted = 0;
  int bestIdx = -1;
  int bestVal = 0;
  for (int i = lo; i <= hi; i++) {
    total += r[i];
    weighted += r[i] * weights[i];
    if (r[i] > bestVal) {
      bestVal = r[i];
      bestIdx = i;
    }
  }

  // If the line is seen, move the window to follow it.
  if (total >= LOST_TOTAL && bestIdx >= 0) {
    lastIdx = bestIdx;
    tracking = true;
    lastSeenTime = millis();
    // An outside sensor firing tells us which way the line went, so the
    // lost-line pivot turns the correct way.
    if (bestIdx == 0) {
      lastSeenDir = -1;
    } else if (bestIdx == 4) {
      lastSeenDir = 1;
    }
  } else if (tracking && millis() - lastSeenTime > REACQUIRE_MS) {
    tracking = false;   // lost for too long: search all sensors again
  }

  // Start boost: ramp up to BOOST_SPEED, hold, then drop to STRAIGHT_SPEED.
  float baseSpeed = STRAIGHT_SPEED;
  float maxSpeed  = MAX_INPUT_SPEED;
  unsigned long elapsed = millis() - startTime;
  if (elapsed < ACCEL_MS) {
    // Phase 1: accelerate from START_SPEED to BOOST_SPEED.
    baseSpeed = START_SPEED + (BOOST_SPEED - START_SPEED) * ((float)elapsed
/ ACCEL_MS);
    maxSpeed  = BOOST_SPEED;
  } else if (elapsed < BOOST_MS) {
    // Phase 2: hold the boost speed.
    baseSpeed = BOOST_SPEED;
    maxSpeed  = BOOST_SPEED;
  }
  // Phase 3 (elapsed >= BOOST_MS): normal STRAIGHT_SPEED, as set above.
```

```
float error;
float left;
float right;

if (total < LOST_TOTAL) {
  // ---------- LINE LOST (usually a sharp corner) ----------
  // Pivot toward the side the line was last seen on:
  // outer wheel drives, inner wheel stops.
  float turnSpeed = min(TURN_SPEED, baseSpeed);
  if (lastSeenDir > 0) {
    // Line was on the right: left wheel drives, right wheel stops.
    left = turnSpeed;
    right = 0;
  } else {
    left = 0;
    right = turnSpeed;
  }
  // Reset PID memory so there is no spike when the line comes back.
  error = lastSeenDir * 2.5;
  lastError = error;
  integral = 0;

} else {
  // ---------- LINE SEEN: normal PID ----------
  error = weighted / total;

  // Remember which side the line was on (ignore tiny values near centre).
  if (error > 0.5) {
    lastSeenDir = 1;
  } else if (error < -0.5) {
    lastSeenDir = -1;
  }

  integral += error * DT;
  integral = constrain(integral, -5, 5);
  float derivative = (error - lastError) / DT;
  lastError = error;

  float correction = Kp * error + Ki * integral + Kd * derivative;

  left  = baseSpeed + correction;
  right = baseSpeed - correction;

  // If the slower wheel would drop below MIN_SPEED (motor deadband),
  // raise both wheels equally so the difference (the steering) is kept.
  float slowest = min(left, right);
  if (slowest < MIN_SPEED) {
```

```
      float shift = MIN_SPEED - slowest;
      left  += shift;
      right += shift;
    }

    left  = constrain(left,  0, maxSpeed);
    right = constrain(right, 0, maxSpeed);
  }

  analogWrite(MOTOR1, (int)left);
  analogWrite(MOTOR2, (int)right);

  //-----------------------------------
  // Printing for debugging and testing
  //-----------------------------------
  static unsigned long lastPrint;
  if (millis() - lastPrint > 200) {
    lastPrint = millis();
    // total, error, left motor, right motor
    Serial1.print(total); Serial1.print(' ');
    Serial1.print(error); Serial1.print(' ');
    Serial1.print(left);  Serial1.print(' ');
    Serial1.println(right);
  }
}
```

Appendix E

Sensor reading data snippet

```
X,CH1,Start,Increment,
Sequence,Volt,-2.995000e+00,5.000000e-03
0,2.40e+00,
1,2.28e+00,
2,2.32e+00,
3,2.20e+00,
4,2.24e+00,
5,2.20e+00,
6,2.20e+00,
7,2.36e+00,
8,2.32e+00,
9,2.40e+00,
10,2.36e+00,
11,2.44e+00,
12,2.40e+00,
13,2.52e+00,
14,2.52e+00,
15,2.44e+00,
16,2.52e+00,
17,2.36e+00,
18,2.44e+00,
19,2.28e+00,
20,2.32e+00,
21,2.12e+00,
22,2.20e+00,
23,1.96e+00,
24,2.00e+00,
25,1.68e+00,
26,1.72e+00,
27,1.24e+00,
28,1.24e+00,
29,5.60e-01,
30,6.00e-01,
31,4.80e-01,
32,4.80e-01,
33,7.20e-01,
34,6.80e-01,
35,1.20e+00,
36,1.16e+00,
37,1.68e+00,
38,1.64e+00,
39,2.12e+00,
```

Sensor graph code

```python
import matplotlib.pyplot as plt
import numpy as np

data = np.loadtxt("reading.csv", delimiter=',', skiprows=50, usecols=[0,1])

Seconds = data[:,0]
Seconds /= 200
Voltages = data[:,1]

axes = plt.axes()
axes.plot(Seconds, Voltages)
axes.set_xlabel("Time [s]")
axes.set_ylabel("Voltage [v]")
```